

Ejercicio 1

Crea un programa para un restaurante de comida rapida, el programa presentara el menú de acuerdo a la lista **menu** y permitira seleccionar una opción de comida hasta que el usuario no desee seleccionar más opciones. El programa debe presentar el total a pagar utilizando la lista **precios** que contiene el precio de cada item del menu. (menu y precios son listas paralelas)

Ejemplo Ejecución:

Menu

hamburguesa 3.00

papas fritas 2.00

hot dog 1.50

canguil 1.00

te helado 0.75

Ingresas tu orden: hamburguesa-2

Deseas algo adicional? S/N s

Menu

hamburguesa 3.00

papas fritas 2.00

hot dog 1.50

canguil 1.00

te helado 0.75

Ingresas tu orden: CANGUIL-1

Deseas algo adicional? S/N n

Total a Pagar: 6.90

NOTA: Considere que los productos vegetarianos tiene un descuento del 10%

clasifiacion=["carnicos","vegetariano","carnicos","vegetariano","bebidas",...] #podrian haber más elementos

menu=["hamburguesa","papas fritas","hot dog","canguil","te helado",...] #podrian haber más elementos

precios=[3.00,2.00,1.50,1.00,0.75] #podrian haber más elementos

Ejercicio 2

En la Universidad se les asigna a los estudiantes que ingresan un código de identificación. Este código consta de 12 dígitos que dan cuenta de la siguiente información:

2017 001 032

AAAA CCC OOO

AAAA: año de ingreso

CCC: código de la carrera

OOO: orden en el que se matriculó

El costo de las matrículas de cada carrera se guarda en lista paralelas como se ve a continuación

codigos = ["001", "002", "003", ...]

nombres = ["COMPUTACION", "TELECOMUNICACIONES", "ALIMENTOS", ...]

precio = [790, 890, 567 ...]

La universidad además aplica un sistema de descuentos de acuerdo con las siguientes condiciones:

Si el estudiante lleva 2 años en la Universidad se le realizará un descuento del 5%; si lleva 3 años, un 15%, y si lleva 4 o más, un 25%.

Así por ejemplo el estudiante con matrícula **2017001032** pertenece a la carrera de computación, ingresó en el año 2017 y fue el estudiante número 32 en matricularse

Cree un programa que al iniciar pida el año actual y a continuación pida el número de matrícula de un estudiante y muestre cuál sería el valor por matrícula de ese estudiante. El programa debe continuar pidiendo valores hasta que el usuario ingresa fin

Ingrese año actual: **2020**

Ingrese número de matrícula: **2017001032**

Estudiante de la carrera de COMPUTACIÓN paga 790

Descuento del 15%

Total a pagar \$671

Ingrese número de matrícula: **2019045032**

Matrícula inválida: No existe carrera

Ingrese número de matrícula: **2019002032**

Estudiante de la carrera de TELECOMUNICACIONES paga 890

No aplica descuento

Total a pagar \$890

Ingrese número de matrícula: **fin**

Nota: Una matrícula es válida si contiene 10 dígitos y la carrera existe

Ejercicio 3

Se tiene una lista de cadenas de caracteres llamada especies con la información de las especies que viven en el bosque. Cada cadena de caracteres tiene el siguiente formato:

Nombre | tipo | Numero especímenes en el bosque.

```
especies = ["gavilan gris|ave|3","raton de campo|roedor|89",  
            "halcon reidor|ave|4","perico catirrojo|ave|13",  
            "oso perezoso|oso|7","jaguar|felino|7",...]
```

Cree un programa que pida un tipo y muestre todas las especies de esa familia que están en peligro de extinción (número de especímenes es menor a 5) y el número especies en peligro de extinción de esa familia

Ingrese tipo de especie: ave

Las especies en peligro de extinción de tipo ave son:

gavilán gris : 3

halcón reidor : 4

Hay 2 aves en peligro de extinción

Ejercicio 1

Usted cuenta con la lista de datos donde cada elemento de la lista está compuesto por **cedula/sexo/altura/peso**.

```
datos= ['0920981344/F/1.58/57.5','0923436512/M/1.68/67','0923122398/M/1.71/68',...]
```

Escriba un programa que calcule y muestre lo siguiente:

1. Peso y estatura promedio de mujeres
2. Peso y estatura promedio de hombres
3. Top 3 de alturas

Ejercicio 2

Dada la siguiente lista:

```
productos= [  
    'parlantes;accesorios/pc;80.89',  
    'parlantes;accesorios/pc;80.89',  
    'armónica;instrumento musical;10.10',  
    'balón de fútbol;deportes;25.0',  
    'teclado;instrumento musical; 210.25',  
    'xbox one;consola;300.1',  
    ...,  
    'caja de bolígrafos;oficina;5.99',  
    'ps4;consola;350.00'  
]
```

La cual describe un conjunto de precios con su categoría y precio en el formato **producto;categoría;precio**

Realice lo siguiente:

1. Cree una lista categorías que contenga las distintas categorías presentes en la lista productos. La lista categorías no debe contener categorías repetidas.
2. Solicite al usuario el ingreso de un nuevo producto a la lista productos. Deberá pedir por teclado al usuario ingresar nombre, categoría y precio del producto. Deberá validar que la categoría se encuentre dentro de las categorías existentes en la lista categorías y que el precio sea un número mayor a 0. El producto deberá ser ingresado en la lista siguiendo el mismo formato **producto;categoría;precio**. En caso de que de no cumplir con los requerimientos se deberá mostrar un mensaje indicando el motivo por el que no se lo pudo agregar.

TicketShow necesita una aplicación en python para la venta de boletos a conciertos, para ello cuenta con tres listas, una con los datos del concierto ("concierto/tipo boleto"), otra con la cantidad de boletos disponibles y otra de precios para cada tipo de boleto.

```
concierto= ["Bad Bunny / General", "Bad Bunny / VIP", "Bad Bunny / Tribuna", "Bad Bunny / Black box", "Shakira / General", "Shakira / VIP", "Shakira / Tribuna", "Shakira / Black box", "Maroon 5 / General", "Maroon 5 / VIP", "Maroon 5 / Tribuna", "Maroon 5 / Black box", "Coldplay / General", "Coldplay / VIP", "Coldplay / Tribuna", "Coldplay / Black box"]

disponibilidad= [50,20,10,15,100,20,50,80,14,63,20,23,65,45,85,24]

precios= [20,30,40,50,30,40,50,60,80,90,100,110,50,70,70,120]
```

Crear la función llamada cantantes_unicos(conciertos) que recibe una lista con la información de los conciertos y retorna una lista de nombres de los cantantes de los conciertos sin repetirse.

La función toda_localidad_disponible(conciertos,disponibilidades,cantantes) recibe la lista de conciertos, la lista de disponibilidades y una lista de cantantes y muestra por pantalla la información de disponibilidad de todos los cantantes.

Nombre del concierto #

Tipo de boleto1: disponibilidad

...

Tipo de boleto n: disponibilidad

La función llamada localidades_disponibles_cantante(conciertos, disponibilidades, cantante) que recibe la lista de conciertos, la lista de disponibilidad y un cantante. La función retorna las localidades y la disponibilidad de entradas del concierto del cantante.

La función vender_boletos(conciertos, disponibilidades, precios, cantante, localidad, cant_boletos) que recibe la lista de conciertos, la lista de disponibilidad, la lista de precios, el nombre de un cantante, una localidad y una cantidad de boletos. La función retornar el total por la venta y la lista de disponibilidad actualizada. Total de la venta = (cantidad de boletos*precio) + IVA.

En el programa principal presente un menú con las siguientes opciones. Dicho menú debe seguir mostrándose hasta que el usuario ingrese la opción 5 de salir, donde el programa termina.

1. Ver conciertos: presenta listado de conciertos

2. Ver todo: presenta todo en la lista de conciertos con su respectiva disponibilidad.

3. Ver localidades: permite ingresar por teclado el nombre de un cantante y presentar las localidades con su disponibilidad.

Nombre del cantante

Tipo de boleto1: disponibilidad

....

Tipo de boleto n: disponibilidad

4. Vender boletos: presenta conciertos disponibles para elegir 1. El usuario ingresará el nombre del concierto y luego presenta localidades con disponibilidad mayor a cero. Luego pedir ingresar la localidad y la cantidad de boletos que desea comprar, validar que la cantidad ingresada sea mayor que cero y menor o igual a la disponibilidad, de ser así registra la venta (disminuye la disponibilidad) y presenta el total de que debe pagar el usuario por la compra.

5. Salir.

Justificar

Implemente la función `justificar(linea, tamaño)` que reciba una línea de texto y un entero. Suponga que la línea está compuesta por dos o más palabras separadas por un espacio en blanco y que su longitud es menor a **tamaño**. Entonces, la función debe completar la longitud de la línea hasta llegar a ser exactamente igual a **tamaño**. Para esto agregue lo más equitativamente posible espacios en blanco entre las palabras de la línea empezando desde la izquierda. Observe que la nueva línea no puede empezar ni terminar con espacios en blanco.

No puede utilizar `while`

Ejemplo

Para el tamaño 61 y la frase:

```
"Este es un buen dia para aprobar fundamentos"
```



Debería retornar:

```
"Este    es    un    buen    dia    para    aprobar    fundamentos"
```



Write an algorithm that takes an array and moves all of the zeros to the end, preserving the order of the other elements.

```
move_zeros([1, 0, 1, 2, 0, 1, 3]) # returns [1, 1, 2, 1, 3, 0, 0]
```

Create a function that returns the lowest product of 4 **consecutive digits** in a number given as a string.

This should only work if the number has 4 digits or more. If not, return "Number is too small".

Example

```
lowest_product("123456789") --> 24 (1x2x3x4)
lowest_product("35") --> "Number is too small"
```

Tema 2

Como asistente de médico, usted tiene la tarea de generar un informe de indicadores a partir de un examen de sangre.

El resultado del examen se lo entregan como una cadena de texto. Los indicadores los puede identificar porque estos siempre estarán en mayúsculas, por ejemplo, INR, WBC, RBC, TA, etc. Todo indicador va seguido de un espacio, luego un número con decimales, seguido de otro espacio en blanco y finalmente las unidades. Al final del resultado se encuentra el nombre del médico.

La cantidad de indicadores puede variar. Los puntos no solo aparecen en los decimales, sino también para separar párrafos o en otras ocasiones como las direcciones de e-mail.

Escriba un programa que nos muestre la información desglosada, el nombre del médico y una recomendación de si el paciente debe ir al endocrinólogo. Un paciente debe ir al endocrinólogo si su nivel de azúcar (BGT), está por encima de los 150 mmol/dL. En caso de dar la recomendación, mostrar doble asterisco en el indicador BGT y la recomendación al final. Para el primer ejemplo de arriba el informe sería:

INFORME DE LABORATORIO

INR	1.25	segundos
BGT	180.12	mmol/dL **
HGB	13	g/dL
ESR	3.2	mm/hora
RBC	4000024.2	cel/uL
TA	1.5	ng/dL
WBC	123233.23	cel/uL

Médico: Juan Pozo

**Su nivel de azúcar es alto, se recomienda ir al endocrinólogo.

#ejemplo de resultado

```
resultado = "Resultado de Laboratorio 'Su Salud' Nombre del paciente: Jose Aimas  
Email del paciente: jose.aimas@gmail.com Resultados del laboratorio: INR 1.25  
segundos BGT 180.12 mmol/dL HGB 13 g/dL ESR 3.2 mm/hora RBC 4000024.2 cel/uL TA 1.5  
ng/dL WBC 123233.23 cel/uL. Los valores de este informe no representan un  
diagnóstico. Firma médico responsable Dr. Juan Pablo Pozo Cardenas"
```

TEMA 4 (20 PUNTOS)

Implemente la función `justificar(línea, tamaño)` que recibe una línea de texto y un entero. Suponga que la línea está compuesta por dos o más palabras separadas por un espacio en blanco y que su longitud es menor a `tamaño`. *En la línea no hay dígitos o símbolos especiales (solo letras y espacios en blanco)*. Entonces, su función debe completar la longitud de la línea hasta llegar a ser exactamente igual a `tamaño`. Para esto agregue **lo más equitativamente posible** espacios en blanco entre las palabras de la línea empezando desde la izquierda. **Observe que la nueva línea no puede empezar ni terminar con espacios en blanco.**

La función debe retornar esta nueva línea.

Ejemplo:

Si se pide justificar a tamaño 61 la frase:
"Este es un buen día para aprobar fundamentos"

La función retornaría:

"Este--- es--- un--- buen-- día-- para-- aprobar-- fundamentos"

#Observe que estamos usando '-' en nuestro ejemplo para representar los espacios en blanco añadidos. En la solución final no se debe usar '-' sino espacios en blanco.

Comparación:

Este es un buen día para aprobar fundamentos

Este es un buen día para aprobar fundamentos

Tema 1 (20 puntos). El Algoritmo de Euclides es considerado el más antiguo y no trivial para encontrar el "máximo común divisor" (mcd) entre dos números a y b .

El paso esencial que garantiza la validez del algoritmo consiste en mostrar que el mcd de a y b es:

- considerar que $a > b$ y $b \geq 0$
- si b es cero, mcd es igual a a
- en otro caso, si $b > 0$, es igual al mcd entre b y el residuo de a dividido por b

Realice una función recursiva `mcdeuclides(a,b)` siguiendo el algoritmo de Euclides, y muestre una prueba de escritorio para $a=15$ y $b=6$.

```
>>> mcd_euclides(15,6)
a: 15 b: 6 residuo: 3
a: 6 b: 3 residuo: 0
3
>>> mcd_euclides(72,16)
a: 72 b: 16 residuo: 8
a: 16 b: 8 residuo: 0
8
>>>
```